Security Testing of LLVM's Intel APX Code Generation: Differential Fuzzing and Adversarial Analysis

MICHAEL SHIRES, University of Virginia, USA

Intel Advanced Performance Extensions (APX) is the largest x86 ISA extension in two decades, adding 16 general-purpose registers and new instruction forms including conditional compares (CCMP), conditional faulting moves (CFCMOV), and paired push/pop (PUSH2/POP2). Compiler support for new ISA features is security-critical: a miscompilation under APX-specific optimization paths can silently alter program semantics, creating vulnerabilities invisible to source-level review. I apply three complementary security testing methodologies to LLVM's APX implementation: (1) static emission analysis across 265 SPEC CPU 2017 source files, (2) adversarial context testing with hand-crafted inputs targeting known compiler-fragile patterns, and (3) IR-level differential fuzzing of 546 randomly generated programs. I find that LLVM's APX code generation is *correct but brittle*: zero semantic mismatches across all fuzzing campaigns, but 52% of adversarial test functions caused APX pattern failures. I also identify a flag fragmentation issue where LLVM's "full APX" flag (`-mapxf`) silently omits conditional faulting, leaving 1,120 CFCMOV instructions and 634 branch eliminations unrealized across SPEC benchmarks.

1 Introduction

Compilers are among the most trusted components in the software supply chain. When an optimization pass silently changes what a program computes, the result is a *miscompilation*: a bug invisible to source-level code review, static analysis, and often standard testing. Miscompilation bugs have led to real-world security vulnerabilities [7, 8], making compiler correctness a security-critical property.

ISA extensions are a particularly high-risk surface for miscompilation. Each new instruction form requires new pattern-matching logic in the compiler backend, new IR-level transformations, and new interactions with existing optimization passes. Intel Advanced Performance Extensions (APX) [2]—shipping in Granite Rapids server processors—is the largest such extension in two decades, adding 16 registers, three-operand non-destructive instructions (NDD), paired push/pop (PUSH2/POP2), compound conditional compares (CCMP), and conditional faulting memory accesses (CFCMOV).

This paper applies three complementary security testing methodologies to LLVM's APX implementation:

(1) **Static emission analysis** across 265 SPEC CPU 2017 source files to quantify which APX instructions LLVM emits and where it leaves opportunities unrealized.
(2) **Adversarial context testing** with 23 hand-crafted functions targeting patterns known to stress compilers: memory operands, volatile accesses, `setjmp/longjmp`, and complex control flow.
(3) **Differential fuzzing** of 546 CSmith-generated programs compiled through three paths (no-APX, APX, APX+CF), testing whether APX-specific optimizations preserve program semantics.

My key findings:

- **Correct but brittle.** Zero semantic mismatches across 546 fuzzed programs, but 52% of adversarial test functions caused APX patterns to silently fail.
- **Flag fragmentation.** `-mapxf` ("full APX") omits conditional faulting (`+cf`), leaving 1,120 CFCMOV instructions and 634 branch eliminations unrealized across SPEC.

Author's Contact Information: Michael Shires, jrs6qe@virginia.edu, University of Virginia, Department of Computer Science, Charlottesville, Virginia, USA.

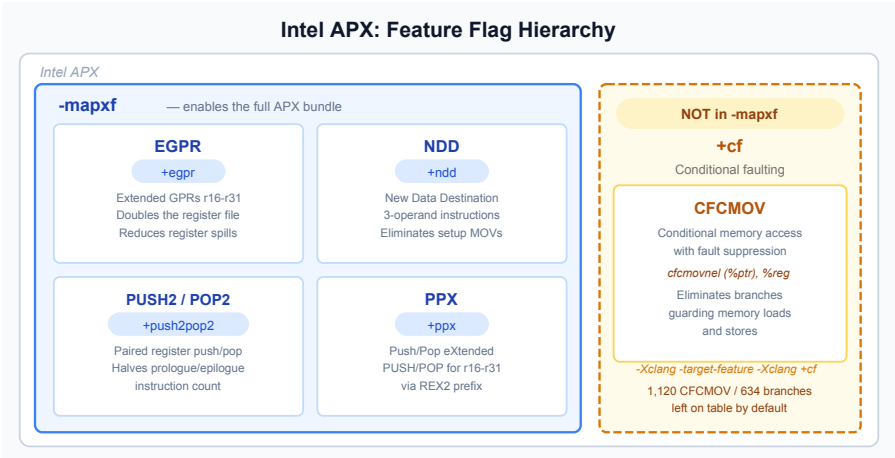

Fig. 1. APX feature flag hierarchy. `-mapxf` enables EGPR, NDD, PUSH2/POP2, and PPX. Conditional faulting (`+cf`), which gates CFCMOV, requires a separate flag.

- **Register-only patterns.** CCMP and NDD CMOV require register operands. Pointer dereferences break these patterns—invisible in scalar-parameter benchmarks.

2 Background

2.1 Intel APX Overview

Intel APX is a bundle of sub-features controlled by separate LLVM feature flags (Figure 1). The user-facing flag `-mapxf` enables four sub-features: **EGPR** (`+egpr`, 16 new general-purpose registers r16–r31), **NDD** (+ndd, three-operand non-destructive instructions), **PUSH2/POP2** (`+push2pop2`, paired register save/restore), and **PPX** (`+ppx`, extended push/pop encoding). Critically, conditional faulting (`+cf`) is *not* included in `-mapxf`. This feature enables CFCMOV—conditional memory access with hardware fault suppression—and requires explicit opt-in via `-Xclang -target-feature -Xclang +cf`.

2.2 Key APX Instructions

CCMP chains comparison instructions: the second comparison executes only if the first meets a condition code, collapsing `SETcc + AND/OR` sequences into a single chain. **CFCMOV** performs conditional memory loads/stores with hardware fault suppression, allowing the compiler to eliminate branches guarding pointer dereferences. Unlike NDD CMOV (register-to-register), CFCMOV operates on memory operands. **PUSH2/POP2** save and restore two callee-saved registers in a single instruction, reducing prologue/epilogue size.

2.3 LLVM Pipeline for APX

APX instructions are generated at different pipeline stages: **CFCMOV** requires an IR-level transformation where SimplifyCFGPass replaces branch-guarded loads with `llvm.masked.load` intrinsics *before* instruction selection. **CCMP** and **NDD CMOV** are matched during instruction selection (SelectionDAG). **PUSH2/POP2** are applied during late machine-level prologue/epilogue insertion. This distinction is security-relevant: IR-level transformations affect *all* downstream passes, while ISel-level patterns are local substitutions.

Table 1. APX instruction counts across 265 SPEC CPU 2017 files (`-O2 -mapxf`, clang 22.1.1).

Benchmark	Files	CCMP	CTEST	NDD	PUSH2	POP2	Total
502.gcc_r	38	129	89	173	609	663	1,663
505.mcf_r	10	3	0	11	21	21	56
525.x264_r	37	142	91	207	453	486	1,379
538.imagick_r	99	344	200	412	1,867	2,154	4,977
557.xz_r	81	33	45	44	218	231	571
Total	**265**	**651**	**425**	**847**	**3,168**	**3,555**	**8,646**

3 Methodology

3.1 Static Emission Analysis

I compiled 265 source files from five SPEC CPU 2017 benchmarks (502.gcc_r, 505.mcf_r, 525.x264_r, 538.imagick_r, 557.xz_r) under three configurations using clang 22.1.1: (1) `-O2` (baseline), (2) `-O2 -mapxf`, and (3) `-O2 -mapxf` with `+cf`. I disassembled object files with `llvm-objdump` and counted APX mnemonics. I also performed a *missed opportunity analysis*: counting compound conditional patterns (`SETcc` chains) and `MOV+CMOV` pairs in the baseline to measure conversion completeness.

3.2 Adversarial Context Testing

CSmith programs use scalar parameters—exactly where APX succeeds. To test the boundaries, I wrote 23 functions combining APX-eligible patterns with: memory operands (pointer dereferences), volatile accesses, `setjmp/longjmp`, switch/computed goto, and inlining into complex callers. Each was compiled with `-O2 -mapxf` (and `+cf` for CFCMOV tests) and inspected for the expected APX instruction.

3.3 Differential Fuzzing

Challenge: No APX hardware. APX processors were unavailable, and Intel SDE cannot run under QEMU on Apple Silicon. I designed an *IR-level differential testing* approach that tests optimization correctness without APX hardware.

Three-path compilation. For each CSmith program: (1) compile without APX to native x86_64 (baseline); (2) compile with `-mapxf` to LLVM IR, strip APX feature attributes, recompile to native; (3) same as (2) with `+cf`. All three are executed and CSmith checksums compared.

The key insight: APX features affect optimization decisions *before* instruction selection. When `+cf` is present, SimplifyCFGPass replaces branch-guarded loads with masked load intrinsics—a transformation that persists after stripping feature flags. Any semantic error (wrong condition, wrong passthrough value) produces a different checksum.

Configuration. CSmith programs were generated with limited complexity (3 functions, depth 2) and `--no-volatiles`—my adversarial testing confirmed volatile blocks all APX patterns. Fuzzing ran in Docker (Ubuntu 24.04, x86_64 via QEMU) with clang-19 and CSmith 2.3.0 across two campaigns (100 + 500 programs).

4 Results

4.1 APX Emission in SPEC CPU 2017

Table 1 shows APX instruction counts across five SPEC benchmarks. PUSH2 and POP2 dominate (77.8% of all APX instructions), consistent with mechanical prologue/epilogue pairing. CCMP and CTEST contribute 1,076 instructions; NDD CMOV accounts for 847.

Table 2. Conversion efficiency. CCMP exceeds 100% by converting branch-based compounds beyond SETcc chains.

Benchmark	SETcc chains	CCMP	MOV+CMOV	NDD
502.gcc_r	129	129	39	173
505.mcf_r	0	3	2	11
525.x264_r	195	142	63	207
538.imagick_r	253	344	289	412
557.xz_r	45	33	16	44
Total	**622**	**651**	**409**	**847**

Table 3. Impact of `+cf`. CFCMOV was entirely absent under the default `-mapxf`.

Instruction	`-mapxf`	`+cf`	Δ
CCMP	651	656	+5
CTEST	425	436	+11
NDD CMOV	847	781	−66
CFCMOV	0	1,120	**+1,120**
PUSH2/POP2	6,723	6,715	−8
Total	**8,646**	**9,708**	**+1,062**

Conversion completeness. I compared APX output against non-APX patterns (Table 2). CCMP achieves 105% conversion—exceeding the SETcc chain count because it also replaces branch-based compound conditionals. NDD CMOV is 100% complete: every `MOV+CMOV` pair (409 total) was converted. PUSH2/POP2 pairing ranges from 48–89%, bounded by structural constraints (odd register counts, frame pointer, alignment) rather than compiler limitations.

4.2 Flag Fragmentation: The Missing CFCMOV

Recompiling with `+cf` reveals a significant gap (Table 3): 1,120 CFCMOV instructions appear across 119 files (45%), eliminating 634 conditional branches. Enabling `+cf` also reduces NDD CMOV by 66—CFCMOV subsumes cases where a separate load plus conditional register move was needed.

The mechanism is an IR-level gate. SimplifyCFGPass queries the target for conditional load/store support before hoisting a branch-guarded load into a masked load intrinsic. Without `+cf`, this check returns false and the branch persists through instruction selection. Figure 2 shows the divergent paths.

4.3 Adversarial Context Testing

Figure 3 summarizes adversarial results: 12 of 23 functions failed to produce the expected APX instruction (52%).

Memory operands break CCMP and NDD CMOV.. When comparison operands are loaded via pointers, CCMP is not emitted. Memory loads create separate basic blocks, and instruction selection does not recognize compound conditionals across block boundaries. Even `restrict` does not help—the issue is structural, not alias-related.

Volatile blocks all APX patterns. Every volatile test failed. Volatile forces memory access on every read, preventing register-based pattern matching and blocking speculative hoisting for CFCMOV.

setjmp forces locals to memory. `setjmp` requires locals to survive non-local jumps, forcing them to memory and killing register-based patterns. PUSH2/POP2 (prologue-level) is unaffected.

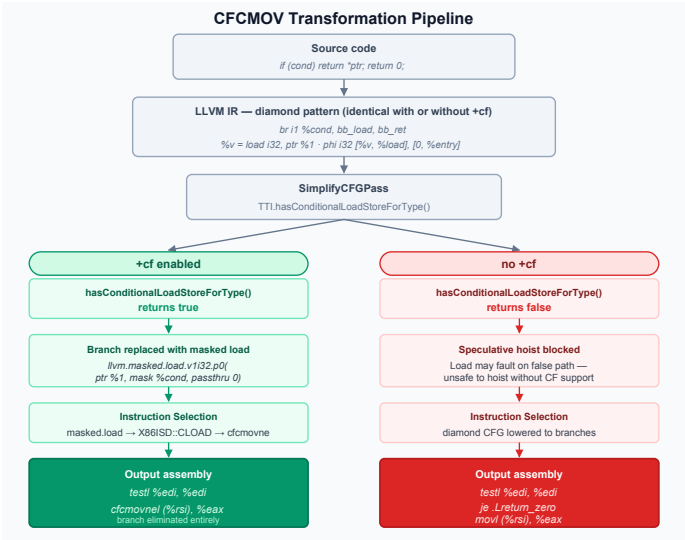

Fig. 2. CFCMOV pipeline. With `+cf`, SimplifyCFGPass replaces branch-guarded loads with `llvm.masked.load`, eventually lowering to `cfcmovnel`. Without `+cf`, the branch persists.

Adversarial Context: APX Pattern Survival

Phase 4 results — 12 of 23 functions failed (52% failure rate)

Pattern Breaker	CCMP	NDD CMOV	CFCMOV	PUSH2/ POP2
Memory operands (pointer dereferences)	BROKEN	BROKEN	OK	N/A
Volatile	BROKEN	BROKEN	BROKEN	N/A
setjmp / longjmp	BROKEN	BROKEN	untested	OK
Switch dispatch	BROKEN	N/A	N/A	N/A
Computed goto	OK	N/A	N/A	N/A
Side effects in condition	OK	N/A	N/A	N/A
Interleaved calls	OK	N/A	OK	OK
Inlining	OK	OK	OK	OK

BROKEN — pattern not emitted · OK — pattern emitted correctly · N/A — instruction not applicable · untested

Fig. 3. Adversarial context results. Memory operands and volatile break CCMP and NDD CMOV. CFCMOV and PUSH2/POP2 are resilient. 12/23 functions failed (52%).

Resilient patterns. CFCMOV survived memory operand tests (its purpose *is* conditional memory access). PUSH2/POP2 survived all applicable tests. All eight inlining test cases passed, confirming late-pipeline patterns survive inlining into complex callers.

Table 4. Differential fuzzing: zero mismatches across 546 CSmith programs compiled through three paths.

Outcome	Initial	Extended	Combined
Checksum match (all 3 paths)	90	456	546
Mismatch (miscompile)	0	0	**0**
Crash (APX only)	0	0	**0**
Programs tested	**90**	**456**	**546**

4.4 Structural Impact

APX affects basic block structure through two mechanisms. EGPR reduces spill instructions, shrinking average block size from 6.21 to 5.78 instructions across all 265 files. CFCMOV merges previously split blocks, raising the average slightly to 5.83 while eliminating 1,123 conditional branches (−2.1%)—reducing branch predictor pressure in compute-intensive loops.

4.5 Differential Fuzzing Results

Table 4 shows fuzzing results. Across 546 completed programs (100 initial + 456 extended), **zero semantic mismatches** were detected. All three compilation paths produced identical CSmith checksums for every completed program.

My IR-level approach tests optimization *decisions* (SimplifyCFG hoisting, CCMP lowering, NDD operand mapping) rather than instruction *encodings*. It does not test actual APX encoding correctness (requiring APX hardware) or detect performance regressions.

5 Discussion

Security implications. LLVM does not produce *wrong* APX code, but silently produces *suboptimal* code under common patterns. Branch-based conditional memory access (without CFCMOV) is vulnerable to speculative execution side channels; CFCMOV eliminates this vector but is silently absent under `-mapxf`. Flag fragmentation creates a false sense of coverage: developers reasonably expect "full APX" from `-mapxf`, unaware that security-relevant optimizations require undocumented flags.

Why memory operands break patterns. LLVM's CCMP matcher operates on register operands within a single basic block. Memory loads may span block boundaries, and the matcher does not recognize compound conditionals across them—even when alias analysis confirms safety. The generated code is correct (using branches), but CCMP's real-world effectiveness is lower than scalar-parameter benchmarks suggest.

Limitations. I tested optimization *decisions*, not APX instruction *execution* (requiring hardware or SDE). CSmith does not cover floating-point, SIMD, or C++ exceptions. The adversarial suite is targeted but not exhaustive.

6 Related Work

Yang et al. [8] introduced CSmith for compiler fuzzing via differential testing. Le et al. [3] extended this with equivalence modulo inputs (EMI). Chen et al. [1] studied GCC/LLVM bugs found through fuzzing. My work applies differential fuzzing specifically to ISA extension code paths.

Wang et al. [7] showed that undefined behavior-based optimizations can introduce security vulnerabilities. CompCert [4] provides formal guarantees but does not support APX. Alive2 [5] verifies LLVM IR transformations but does not extend to backend instruction selection. Prior ISA extension testing has focused on SIMD/AVX [6]; to my knowledge, this is the first security-oriented characterization of LLVM's APX code generation.

7 Conclusion

I applied static emission analysis, adversarial context testing, and differential fuzzing to LLVM's Intel APX code generation. LLVM's APX implementation is *correct but incomplete*: zero semantic mismatches across 546 fuzzed programs, but 52% of adversarial contexts caused silent pattern failures, and the default "full APX" flag leaves 1,120 conditional faulting instructions unrealized. ISA extension support is a security-relevant compiler surface that warrants systematic testing beyond standard regression suites.

References

[1] Yang Chen, Alex Groce, Chaoqiang Zhang, Weng-Keen Wong, Xiaoli Fern, Eric Eide, and John Regehr. 2013. Taming Compiler Fuzzers. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 197–208. doi:10.1145/2491956.2462173

[2] Intel Corporation. 2024. *Intel Advanced Performance Extensions (APX) Architecture Specification*. https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-performance-extensions-apx.html Revision 3.0.

[3] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler Validation via Equivalence Modulo Inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 216–226. doi:10.1145/2594291.2594334

[4] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115. doi:10.1145/1538788.1538814

[5] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 65–79. doi:10.1145/3453483.3454030

[6] Anirudh Taneja, Sungmin Kim, and John Regehr. 2020. Testing Static Analyses for Precision and Soundness. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE.

[7] Xi Wang, Haogang Chen, Alvin Cheung, Zhihao Jia, Nickolai Zeldovich, and M. Frans Kaashoek. 2012. Undefined Behavior: What Happened to My Code?. In *Proceedings of the Asia-Pacific Workshop on Systems (APSys)*. ACM. doi:10.1145/2349896.2349905

[8] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 283–294. doi:10.1145/1993498.1993532